

One bag, one day

Michael — you decided two things before this slice, and both of them are now built into the books rather than living in a conversation.

You said: *"For question 1, post with a warning."* And you said: *"I don't want to include the home safe. I will keep cash at the shop and start doing daily deposit bags. Even if I don't make it to the bank for 15 days, each deposit will still match the day it came from."*

What changed

Before this slice, undeposited cash sat in one lump. A \$110,000 deposit covering fifteen days cleared against a single total, and the books could not say which days it emptied. Now the pool is kept **day by day**. When the bank confirms a deposit, the entry pays off the oldest open day first, then the next, until the money runs out — writing **one credit line per day, with that day named on the line**. One bank credit now points at named Z-report days instead of at a nameless balance.

The end-of-day screen changed to match the safe. Instead of one number, it lists the open days one at a time with what each is worth, so a manager can hold the list against the bags and count. That is the point of the daily bag procedure: the paper and the screen should be the same shape.

Cash older than thirty days used to be **refused**. It now **posts, with a warning** that rides on the entry itself, not just a screen message that dies when the page closes. Your reasoning was right: refusing the entry does not make the money younger, it leaves real cash unrecorded, which is worse.

The mistake this slice made, and the rule it produced

My first version of the day-by-day pool did not work, and its own test caught it. A clearing entry is dated the day the **bank** saw the money, not the day the **sale** happened. Group the pool by entry date and a banked day never cancels itself out — it stays open forever, and the exact bug I was fixing survives the fix, quietly.

The fix was to stamp each clearing line with the business day it retires, and read that stamp back. That produced a new standing rule: **a fix that restates the bug's own assumption is not a fix**. It now applies to everything after this.

Two more real problems surfaced. One was a crash: a deposit against a pool with a missing column would have thrown a 500 error on your screen. That is fixed in the code, not papered over in the test. The other was a test that was lying — it handed back the day-stamp even when the query never asked for it, so the whole day-naming feature could have been deleted with every test still passing. Both are closed, with a deliberate break guarding each.

What I did not build, on purpose

There is **no home safe account**, because you told me not to add one, and adding accounts money never sits in is how books stop matching life. That refusal is written into the code where a future developer will read it.

The deposit is still tied to business **days**, not named register **sessions**. Two tills closing on the same day fold together, because the bank feed cannot tell me how the bag was composed. That is honest rather than fixed, and recorded as such in the census.

"Oldest first" is a convention, not a fact. Your bags are physical and could be banked out of order. The books pick the oldest open day because that is standard and it keeps the pool from ageing forever — but it is a choice, and says so where it is made.

Where things stand

501 test files, 12,824 tests, all passing — 35 more than last slice. The deposit file alone now carries 89 tests. TypeScript is clean, lint is clean, the census document matches the code, and every quoted rule still matches its official source.

I also ran two sets of deliberate sabotage: 36 breaks aimed at last slice's work and 36 at this slice's. All 72 were caught. Five attack the **screen** rather than the math — deleting the day list, hiding the amounts, claiming all-clear when the pool is over-cleared — because a correct number nobody can see is not a working feature.

Defect count is unchanged at 76, but **D-76 is now closed** rather than open. Nothing new was added to the list.